

LC 72: Edit Distance

Authored by: Arpit Raj, LNMIIT Jaipur

October 1, 2026

Deloitte, Swiggy, TCS, Flipkart, Visa

Problem Statement

Given **word1** and **word2**, find the minimum number of operations required to convert *word1* → *word2*.

Allowed Operations: Insertion, Deletion, Replacement

Example 1

"horse" → "ros"

horse → rorse (replace 'h' with 'r')

rorse → rose (delete 'r')

rose → ros (delete 'e')

Example 2

"aadya" → "33dya"

Output = **2** (and not 1)

Because both the 'a's are considered different and need separate replacements.

Classical 2D DP Problem

DP State:

$dp[i][j]$ → min no. of ops required to convert first i letters of $w1$ to first j letters of $w2$.

Base Case 1

$$dp[i][0] = i$$

(cause deleting all)

Base Case 2

$$dp[0][j] = j$$

(cause inserting all)

 Handwritten notes page 1

Reference Note 1: Setup, Examples, and DP State

Transitions & Conditions

If 2 letters are same, no ops required:

$$dp[i][j] = dp[i-1][j-1]$$

If not same (when last characters are not matching):

- $dp[i][j] = 1 + dp[i-1][j]$ (deletion)
- $dp[i][j] = 1 + dp[i][j-1]$ (insertion)
- $dp[i][j] = 1 + dp[i-1][j-1]$ (replace)

Visualizing the Operations

Eg 1 (Deletion):

w1: a b **c** (i)

w2: a b (j) → so delete 'c'

$$dp[i][j] = 1 + dp[i-1][j]$$

1 for delete, $dp[i-1][j]$ for solving prev cost

Eg 2 (Insertion):

w1: a b (i)

w2: a b **c** (j)

$$dp[i][j] = 1 + dp[i][j-1]$$

1 for insertion, $dp[i][j-1]$ for solving prev cost

Eg 3 (Replacement):

w1: ad**y** (i)

w2: ad**z** (j)

$$dp[i][j] = 1 + dp[i-1][j-1]$$

1 make equal, $dp[i-1][j-1]$ solve prev cost

C++ Implementation

Full code representing the 2D DP transition logic defined in the notes.

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int n1=word1.size();
        int n2=word2.size();

        vector<vector<int>> dp(n1+1,vector<int>(n2+1,INT_MAX));
        for(int i=0;i<=n1;i++){
            dp[i][0]=i;
        }

        for(int j=0;j<=n2;j++){
            dp[0][j]=j;
        }

        for(int i=1;i<=n1;i++){
            for(int j=1;j<=n2;j++){

                if(word2[j-1]==word1[i-1]){
                    dp[i][j]=dp[i-1][j-1];
                }

                else{
                    int c1=1+dp[i-1][j]; //deletion
                    int c2=1+dp[i][j-1]; //insertion
                    int c3=1+dp[i-1][j-1]; //replace

                    dp[i][j]=min({dp[i][j],c1,c2,c3});
                }
            }
        }

        return dp[n1][n2];
    }
};
```